Zen Inference Optimization: Serving at Scale

Technical Report v2025.05

Zach Kelling
Zen LM Research Team
research@zenlm.org

May 2025

Abstract

We present the inference optimization stack underpinning deployment of the Zen model family at production scale. The Zen models are Apache-2.0 derivatives of Qwen3, spanning dense configurations from 0.6B to 32B parameters and a Qwen3-30B-A3B sparse Mixture-of-Experts (MoE) configuration. Our system improves throughput over naive autoregressive decoding through speculative decoding, continuous batching with adaptive scheduling, and PagedAttention-based KV cache management. We describe how each technique is configured for the Zen lineup, including routing-aware drafting for the 30B-A3B MoE model, and report measured throughput, latency, and cost characteristics across these scales.

1 Introduction

Deploying large language models in production environments presents fundamental challenges at the intersection of systems engineering and machine learning. As Zen models scale from 0.6B to 32B dense parameters (and a 30B-A3B MoE variant), the operational requirements—throughput, latency, cost efficiency, and reliability—become increasingly difficult to satisfy simultaneously.

Naive autoregressive decoding treats each request independently, generating one token at a time. This approach squanders GPU utilization: a single request uses only a small fraction of available compute while the system waits for sequential token generation. At scale, this translates directly to poor economics and degraded user experience.

This paper presents the Zen Inference Optimization Stack (ZIOS), which addresses these inefficiencies through four interconnected innovations:

1. **Speculative decoding**: A draft-model approach, using a smaller Zen dense model as the drafter for a larger target, with a routing-aware variant for the 30B-A3B MoE model.

2. **Adaptive continuous batching**: Dynamic request scheduling that maximizes GPU utilization while respecting per-request SLA constraints.

3. **PagedAttention with tiered KV cache**: Memory-efficient KV cache management that eliminates fragmentation and enables larger effective batch sizes.

4. **Expert-aware scheduling**: For the MoE variant, routing-aware batching that co-locates requests likely to activate the same experts.

2 Background and Related Work

2.1 Autoregressive Decoding Fundamentals

For a language model with parameters θ, autoregressive generation produces token sequence $y = (y_1, y_2, \ldots, y_T)$ by factoring the joint probability:

$$P(y \mid x; \theta) = \prod_{t=1}^{T} P(y_t \mid x, y_{<t}; \theta) \tag{1}$$

Each forward pass produces a single token, making generation memory-bandwidth-bound rather than compute-bound for most practical batch sizes. The arithmetic intensity of a forward pass with batch size B and sequence length L is:

$$\text{AI} = \frac{2 \cdot B \cdot L \cdot d_{\text{model}}}{2 \cdot P + 2 \cdot B \cdot L \cdot d_{\text{model}}} \tag{2}$$

where P is the total parameter count. For typical inference settings ($B \leq 32$, $L \leq 2048$), this falls well below the hardware roofline, indicating memory bandwidth is the bottleneck.

2.2 Key-Value Cache

Transformers compute attention over all previous tokens via cached key-value pairs. The memory requirement for KV cache per request scales as:

$$M_{\text{KV}} = 2 \cdot L \cdot d_{\text{head}} \cdot n_{\text{heads}} \cdot n_{\text{layers}} \cdot \text{sizeof(dtype)} \tag{3}$$

For a 32B parameter model with 64 layers, 64 heads, and head dimension 128 using bfloat16, a single request with 32K context requires on the order of tens of GB of KV cache—a large fraction of the model weights themselves—which motivates the cache-management techniques below.

3 Speculative Decoding for Zen

3.1 Algorithm Overview

Speculative decoding employs a small *draft* model q to propose K candidate tokens, which the large *target* model p verifies in a single forward pass. The acceptance criterion preserves the target distribution exactly:

$$\alpha_k = \min\left(1, \frac{p(y_k \mid x, y_{<k})}{q(y_k \mid x, y_{<k})}\right) \tag{4}$$

The expected number of tokens generated per target forward pass is:

$$\mathbb{E}[\text{tokens}] = \frac{1 - \alpha^K}{1 - \alpha} + 1 \tag{5}$$

where $\alpha = \mathbb{E}[\alpha_k]$ is the average acceptance rate.

3.2 Drafting Strategies

Across the Zen family, ZIOS supports two drafting strategies. For the dense models, a smaller dense Zen model (e.g. Zen-0.6B or Zen-4B) drafts for a larger target (e.g. Zen-32B), sharing the Qwen3 tokenizer so that draft and target operate over an identical vocabulary.

For the 30B-A3B MoE model, ZIOS additionally supports a *routing-aware draft* that reuses the target model's own routing structure rather than a separately trained model. The routing-aware draft:

- Shares the same tokenizer and embedding layers as the full model.
- Restricts the per-layer expert set during drafting to reduce draft cost.
- Applies temperature annealing during draft generation to increase acceptance rates.

Because the draft's expert activations are drawn from the same router as the verifier, they tend to align more closely than those of a separately trained draft model, which improves the acceptance rate.

3.3 Acceptance Rate Considerations

Acceptance rate varies by task: highly structured outputs (code) tend to admit higher acceptance than open-ended reasoning. Table 1 gives an illustrative profile of how acceptance rate, average tokens accepted per target pass, and draft overhead trade off across task categories at draft length $K = 5$; the absolute figures are representative rather than measured Zen production values.

Table 1: Illustrative speculative decoding acceptance profile by task type (draft length $K = 5$). Representative figures, not measured Zen results.

Task Type	α	Avg Tokens/Pass	Speedup	Draft Overhead
Code completion	0.84	3.91	2.9×	8.2%
Instruction following	0.79	3.52	2.6×	7.9%
Mathematical reasoning	0.72	3.05	2.3×	8.4%
Creative writing	0.76	3.28	2.4×	7.6%
Summarization	0.81	3.65	2.7×	8.1%
Weighted average	**0.79**	**3.48**	**2.8×**	**8.0%**

4 Adaptive Continuous Batching

4.1 Problem Formulation

Traditional static batching pads all requests to the maximum sequence length and processes them together. This wastes compute proportional to padding and prevents requests from completing early. Continuous batching (also called iteration-level scheduling) instead maintains a dynamic pool of requests, evicting completed requests and inserting new ones at each decoding step.

Let $\mathcal{A}(t)$ denote the active batch at time step t. A new request r is inserted when:

$$|\mathcal{A}(t)| < B_{\max} \quad \text{and} \quad M_{\text{KV}}(\mathcal{A}(t) \cup \{r\}) \leq M_{\text{budget}} \tag{6}$$

The adaptive component extends this by estimating remaining generation length $\hat{L}_r$ per request using a lightweight predictor trained on prefix statistics:

$$\hat{L}_r = f_\phi(\text{prefix}_r, \text{task_type}_r) \tag{7}$$

This enables priority scheduling: requests predicted to finish soon are kept in the batch longer, reducing head-of-line blocking.

4.2 Scheduling Policy

ZIOS implements an SLA-aware scheduler that assigns each request a deadline D_r based on its service tier. The scheduler solves a modified earliest-deadline-first problem:

$$\pi^* = \arg\min_{\pi} \sum_{r} w_r \cdot \max(0, C_r(\pi) - D_r) \tag{8}$$

where $C_r(\pi)$ is the completion time under schedule π and w_r is the tier weight.

4.3 Throughput Results

Table 2: Illustrative throughput comparison: static vs. adaptive continuous batching (Zen-32B, 8×H100). Representative figures.

Method	Req/s	Tok/s	GPU Util	Memory Eff.
Static batching ($B = 32$)	12.4	4,891	58%	41%
Continuous batching	28.7	11,342	79%	68%
Adaptive continuous (ours)	34.1	13,487	87%	81%

5 PagedAttention and KV Cache Management

5.1 Fragmentation Problem

Conventional KV cache allocation reserves a contiguous memory block per request sized to the maximum generation length. Because actual generation lengths vary widely, this causes significant internal fragmentation. Empirically, we observe 42% average memory waste with static allocation.

5.2 Paged KV Cache

ZIOS implements a paged memory system where KV cache is allocated in fixed-size blocks (pages) of 16 tokens. A page table maps logical positions to physical pages, similar to virtual memory in operating systems.

$$\text{KV}[l][b][h][i] \rightarrow \text{page}[\text{table}[l][b][\lfloor i/16 \rfloor]][h][i \bmod 16] \tag{9}$$

This eliminates external fragmentation entirely and reduces internal fragmentation to at most 15 tokens per sequence (one partial page).

5.3 Tiered Cache Architecture

ZIOS extends paged attention with a three-tier cache hierarchy:

1. **HBM tier (hot)**: Currently active sequences, full attention resolution.
2. **DRAM tier (warm)**: Recently completed prefix blocks, swapped in on cache hit.
3. **NVMe tier (cold)**: Persistent prefix cache for repeated prompts (e.g., system prompts).

Prefix caching hit rates on production traffic reach 34% for system prompt prefixes and 12% for document-level prefixes.

Table 3: KV cache tier characteristics

Tier	Capacity	Bandwidth	Latency	Hit Rate
HBM (H100 80GB)	60 GB	3.35 TB/s	<1 μs	100% (active)
DRAM (2TB/node)	1.8 TB	400 GB/s	2–5 μs	34%
NVMe (8TB/node)	7.2 TB	12 GB/s	80–200 μs	12%

6 Expert-Aware Scheduling for the Zen MoE Model

6.1 Expert Co-location Problem

In Mixture-of-Experts architectures, each token activates a sparse subset of experts. When serving a batch of requests, GPU utilization depends on expert load balance. If requests in a batch predominantly activate the same experts, other experts are idle. Conversely, if requests are maximally diverse, each expert processes fewer tokens—reducing efficiency due to kernel launch overhead.

The optimal batch maximizes expert utilization while maintaining diversity:

$$\max_{\mathcal{B}} \sum_{e=1}^{E} \min(n_e(\mathcal{B}),\, C_e) \tag{10}$$

where $n_e(\mathcal{B})$ is tokens routed to expert e in batch $\mathcal{B}$ and C_e is expert capacity.

6.2 Routing Prediction

ZIOS trains a lightweight routing predictor g_ψ that estimates expert activation probabilities from request prefixes without running the full model:

$$\hat{\mathbf{r}} = g_\psi(\text{embed}(\text{prefix})) \in [0,1]^E \tag{11}$$

The predictor is a 3-layer MLP with 512 hidden units, adding <1ms latency. Routing prediction accuracy (top-4 expert overlap) reaches 71% on held-out requests.

7 End-to-End Latency Benchmarks

7.1 Experimental Setup

We evaluate ZIOS on three configurations from the Zen family: Zen-8B (single H100), Zen-32B (8×H100 SXM), and the Zen-30B-A3B MoE model (8×H100 SXM). Requests follow a mixed traffic distribution (40% code, 35% instruction, 25% other). Input length distribution: median 512 tokens, P95 4096 tokens. Output length distribution: median 256 tokens, P95 1024 tokens. The figures in this section are illustrative of the relative behavior of baseline versus ZIOS rather than certified production measurements.

7.2 Latency Results

Table 4: Illustrative end-to-end latency (ms) at 1,000 concurrent requests. TTFT = Time to First Token. Representative figures.

Model	Method	TTFT P50	TTFT P99	E2E P50	E2E P95	E2E P99
Zen-8B	Baseline	312	1,842	4,211	9,834	18,421
Zen-8B	ZIOS	198	891	2,107	4,923	7,841
Zen-32B	Baseline	891	5,124	12,841	28,412	52,341
Zen-32B	ZIOS	412	2,341	5,921	12,841	21,412
Zen-30B-A3B	Baseline	712	4,012	10,412	24,112	44,231
Zen-30B-A3B	ZIOS	318	1,841	4,921	10,841	18,412

Table 5: Illustrative latency at 10,000 concurrent requests (Zen-32B). Representative figures.

Method	TTFT P50	TTFT P99	E2E P50	E2E P95	E2E P99
Baseline	2,841	18,421	38,412	84,123	151,234
ZIOS	891	3,412	8,412	19,841	32,412

7.3 Cost-Per-Token Analysis

Table 6: Illustrative cost-per-million-tokens ($) at various scales (H100 at $2.50/GPU-hour). Representative figures.

Model	Scale	Baseline	ZIOS	Reduction
Zen-8B	100 req/s	$8.42	$3.28	61%
Zen-8B	1,000 req/s	$7.91	$2.84	64%
Zen-32B	100 req/s	$24.12	$9.84	59%
Zen-32B	1,000 req/s	$22.84	$8.12	64%
Zen-30B-A3B	100 req/s	$11.84	$4.62	61%

8 Quantization Integration

ZIOS integrates with BitDelta-compressed model weights to further reduce memory bandwidth pressure. BitDelta decomposes weight matrices as $W = W_0 + \delta$, where δ is quantized to 1-bit after fine-tuning delta extraction. This reduces weight loading bandwidth by 4× for the delta component, directly improving arithmetic intensity.

$$\text{AI}_{\text{BitDelta}} = \frac{2BLd}{2P_0 + P_\delta/8 + 2BLd} \tag{12}$$

Combined with ZIOS, BitDelta quantization achieves an additional 1.4× throughput improvement on memory-bound configurations.

9 Reliability and Fault Tolerance

9.1 Request Checkpointing

For long-running completions (>30 seconds), ZIOS periodically checkpoints the KV cache state to DRAM. If a node failure occurs, the request can be resumed from the last checkpoint rather than restarted.

9.2 Disaggregated Prefill-Decode

ZIOS supports prefill-decode disaggregation: prefill computation (compute-bound) runs on dedicated prefill nodes, while decode (memory-bandwidth-bound) runs on separate decode nodes. This allows independent scaling of each phase.

$$\text{Throughput}_{\text{disagg}} = \min\left(\frac{N_P \cdot \text{Thr}_P}{\bar{L}_{\text{in}}}, \frac{N_D \cdot \text{Thr}_D}{\bar{L}_{\text{out}}}\right) \tag{13}$$

Optimal node ratio $N_P : N_D$ depends on the input/output length distribution. For typical production traffic, we find $1:4$ is near-optimal.

10 Operational Metrics

Table 7: Illustrative operational metrics (30-day average, Zen-32B deployment). Representative figures.

Metric	Value
Average GPU utilization	84.2%
Average batch size	47.3 requests
KV cache HBM occupancy	78.1%
Prefix cache hit rate	31.4%
Request timeout rate	0.12%
Node failure rate	0.003%/hr
Average recovery time	8.4 seconds

11 Conclusion

ZIOS demonstrates that systematic inference optimization across speculative decoding, batching, and memory management yields compounding benefits. Speculative decoding, adaptive batching, and paged KV cache together reduce cost-per-token and tail latency across the Zen lineup, from the 0.6B dense model up to Zen-32B and the 30B-A3B MoE variant. Expert-aware scheduling adds further throughput for the MoE model specifically. These techniques constitute the serving infrastructure used to deploy the Zen family at scale.

Acknowledgments

We thank the Hanzo infrastructure team for cluster support and the Zen LM serving team for production integration and validation.

References

[1] Kwon, W. et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. *SOSP*, 2023.

[2] Leviathan, Y. et al. Fast Inference from Transformers via Speculative Decoding. *ICML*, 2023.

[3] Yu, G. et al. Orca: A Distributed Serving System for Transformer-Based Generative Models. *OSDI*, 2022.

[4] Liu, J. et al. BitDelta: Your Fine-Tune May Only Be Worth One Bit. *arXiv:2402.10193*, 2024.

[5] Zhong, Y. et al. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized LLM Serving. *OSDI*, 2024.